

Strategic Product Placement Analysis

Unveiling Sales Impact with Tableau Visualization

Full Stack Application — Project Documentation

Team ID: LTVIP2026TMIDS55759

Team: Neriyanuru Vedapriya, Pavan Kumar Pola, Masetty Suvarna Linga Lakshmi Kanth, Mula Gopi

College: Kallam Haranadha Reddy Institute of Technology, Guntur

Date: June 2026

1. Project Title

Strategic Product Placement Analysis: Unveiling Sales Impact with Tableau Visualization — extended into a full-stack web application with ML-based prediction.

2. Problem Statement

Retail managers and analysts lack a centralized, visual, and interactive way to understand how product placement strategy (aisle, end-cap, front-of-store), promotions, foot traffic, seasonality and consumer demographics influence sales volume. Existing data is scattered across static reports, making fast, confident, data-driven placement decisions difficult.

3. Objectives

- Centralize sales, pricing, promotion and placement data in a relational database.
- Provide an interactive web dashboard summarizing placement performance.
- Provide a machine-learning powered 'what-if' predictor for sales volume given a placement scenario.
- Provide secure, role-based access via REST APIs and JWT authentication.
- Preserve the original Tableau dashboards/stories as the executive analytics layer.

4. System Architecture

The system follows a 3-tier architecture: a static HTML/CSS/JS frontend, a Flask REST API backend, and a MySQL relational database. A scikit-learn model is trained offline from the cleaned dataset and served through the prediction API. Tableau Public hosts the executive dashboards/stories built from the same source data.

- Presentation Layer: Responsive HTML/CSS/JS SPA-style frontend with Chart.js visual widgets.
- Application Layer: Flask REST API (auth, data/analytics, prediction blueprints) secured with JWT.
- Data Layer: MySQL database (users, products, sales_records, predictions, feedback).
- ML Layer: Offline-trained RandomForestRegressor, served via pickle-loaded predict.py.
- Visualization Layer: Tableau Public dashboards (7 visuals) and story (3 visuals) consuming the same dataset.

5. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript, Chart.js
Backend	Python, Flask, Flask-JWT-Extended, Flask-CORS
Database	MySQL 8.0, SQLAlchemy ORM
Machine Learning	scikit-learn (RandomForestRegressor), pandas, numpy
Visualization	Tableau Desktop / Tableau Public
Deployment	Docker, Docker Compose, Gunicorn, Render/Cloud

6. Database Design

Relational schema (MySQL):

- users(user_id PK, username, email, password_hash, role, created_at)
- products(product_id PK, product_code, product_category, price, competitor_price)
- sales_records(record_id PK, product_id FK, product_position, promotion, foot_traffic, consumer_demographic, seasonal, sales_volume, record_date)
- predictions(prediction_id PK, user_id FK, product_category, product_position, promotion, foot_traffic, seasonal, predicted_sales_volume, created_at)
- feedback(feedback_id PK, user_id FK, message, created_at)

Full DDL is provided in database/schema.sql.

7. Workflow Diagram (Data Flow)

Retail Manager -> Tableau / Web App -> Data Preparation -> Product Placement & Sales Performance Dashboard -> Dashboard Insights stored back to Sales Data, mirroring the original DFD from the ideation phase.

- Raw sales/placement data is collected from POS/ERP/CRM systems.
- Data is cleaned (duplicates removed, missing values handled) in data preparation step.
- Cleaned data feeds both the MySQL database (web app) and Tableau extracts (dashboards).
- The ML model consumes the same cleaned features to predict sales volume for new scenarios.

8. API Documentation

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user	No
POST	/api/auth/login	Login, returns JWT	No
GET	/api/auth/me	Current user profile	Yes
GET	/api/data/products	List products	Yes
GET	/api/data/sales	List/filter sales records	Yes
GET	/api/data/analytics/summary	Avg sales by category	Yes
GET	/api/data/analytics/by-position	Avg sales by placement	Yes
GET	/api/data/analytics/foot-traffic	Avg sales by foot traffic	Yes
POST	/api/predict/sales	Predict sales volume	Yes
GET	/api/predict/history	User prediction history	Yes

9. ML Model Explanation

Goal: predict Sales Volume given a placement scenario (category, position, promotion, foot traffic, demographic, seasonality, price, competitor price).

- Preprocessing: duplicate removal, missing-value forward-fill, label encoding of categorical fields.
- Algorithm: RandomForestRegressor (200 estimators, max depth 10, random_state=42) — chosen for robustness to mixed categorical/numeric features and resistance to overfitting on a moderate-size retail dataset.
- Evaluation Metrics: Mean Absolute Error (MAE) and R-squared (R2), printed at training time (train_model.py).
- Serving: model and encoders are pickled (sales_model.pkl, encoders.pkl) and loaded once by predict.py; the /api/predict/sales endpoint logs every prediction to the predictions table for traceability.

10. Tableau Dashboard Explanation

The Tableau dashboard ('Product Placement Analysis') contains 7 visualizations: Avg Sales Volume vs Product Category (bar), Competitor Price vs Price (side-by-side bar), Avg Sales Volume by Category by Position (tree map), Consumer Demographics vs Sales Volume (donut), Product Category vs Price (pie), Avg Sales by Season by Category (stacked bar), and Foot Traffic by Avg Sales (packed bubbles). A companion Story walks through 3 key insights: Clothing has the highest average sales volume; Front-of-Store position correlates with high sales volume and foot traffic; Clothing at Front-of-Store yields the highest average sales volume overall.

11. Installation Guide

- Clone or download the project folder (SPPA/).
- Create the MySQL database: `mysql -u root -p < database/schema.sql`
- Backend: `cd backend`, create venv, `pip install -r requirements.txt`, copy `.env.example` to `.env` and edit credentials.
- Train the ML model: `python app/ml/train_model.py --file`
- Seed data: `python database/seed_data.py --file`
- Run backend: `python run.py` (serves on port 5000).
- Frontend: open `frontend/index.html` or serve via `python -m http.server 8080`.

12. Deployment Guide

- Docker: `cd deployment && docker-compose up --build` (spins up MySQL, Flask backend, and Nginx-served frontend).
- GitHub: `git init`, `git add .`, `git commit`, create remote repo, `git push`.
- Cloud: `deployment/render.yaml` provides a ready Render.com config (build/start commands, env vars); the same pattern (build command + start command + env vars) applies to Azure App Service, AWS Elastic Beanstalk, or Heroku.

13. Future Enhancements

- Real-time POS data ingestion and live dashboard refresh.
- What-if simulation UI directly tied to the ML model with confidence intervals.
- Role-based dashboards for Sales Executives, Marketing Analysts and Stakeholders.
- In-store heatmap / footfall sensor integration.
- Cross-channel (online + offline) placement analytics.

14. Conclusion

This package extends the original Tableau-based capstone project into a complete, production-style full-stack system: a secure Flask/MySQL backend, a responsive web frontend, and a trained machine learning model for sales-volume prediction, while preserving the original Tableau dashboards and story as the executive visualization layer. The architecture is modular, containerized, and ready for local execution, GitHub version control, or cloud deployment.